

TP3 Robot collecteur

Ennemis, boucle de jeu et rétroaction audiovisuelle

Durée 3 heures	Version Unity 6000.2.5f1	Point de départ Prototype complet des TP1 et TP2
--------------------------	------------------------------------	--



Aperçu du résultat attendu : scène industrielle, HUD, batteries, obstacles, ennemis et porte.

Mission du TP

Vous recevez un prototype qui regroupe les fonctionnalités réalisées dans les TP1 et TP2. Votre travail consiste à enrichir **la scène existante**. Vous ne devez pas créer une nouvelle scène ni remplacer le décor fourni.

À la fin du TP, deux ennemis se déplacent de manière différente, détectent le robot et déclenchent une rétroaction lors d'un contact. Vous devez également construire l'interface du jeu afin d'afficher les batteries, les vies, le temps et les états de victoire ou de défaite.

Objectifs d'apprentissage

- Ajouter des objets interactifs dans une scène Unity déjà construite.
- Programmer deux stratégies de déplacement avec un même script.
- Utiliser Rigidbody2D, Collider2D et les événements de déclenchement.
- Communiquer avec GestionJeu pour retirer une vie et terminer la partie.
- Construire une interface adaptable et relier ses éléments aux scripts du jeu.
- Créer une courte introduction cinématique avant le début de la partie.
- Créer une rétroaction claire avec couleur, changement d'échelle, son et secousse de caméra.

- Tester une boucle complète : jouer, réussir, échouer et recommencer.

Organisation du travail et versions Git

Travaillez dans le dépôt GitHub commencé au TP1. Chaque fonctionnalité doit faire l'objet d'un commit distinct. Ne déposez pas tout le projet dans un seul commit final.

Étape	Commit attendu	Résultat vérifiable
Préparation	Préparer la scène du TP3	La scène TP1 et TP2 est intacte et jouable.
Ennemis	Ajouter les ennemis et leurs trajets	Deux ennemis correctement dimensionnés sont visibles.
Déplacements	Programmer les déplacements différents	Un ennemi patrouille et l'autre se déplace en vague.
Interactions	Ajouter poursuite et impact	Le robot perd une vie et retourne au départ.
Rétroaction	Ajouter effets attaque et audio	L'impact est visible et audible.
Interface	Construire le HUD, l'introduction et les écrans de fin	Les informations et les états du jeu sont présentés clairement.
Finition	Tester la boucle complète	Victoire, défaite et redémarrage fonctionnent.

Ressources fournies

- Le prototype Unity réunissant les solutions des TP1 et TP2.
- Le sprite de l'ennemi et les ressources graphiques du jeu.
- Les images du logo, du cœur, de la batterie, du chronomètre et du parchemin pour l'interface.
- Les scripts de départ ou des lignes de code à remettre en ordre, selon la version distribuée.
- Une solution enseignant comportant un outil Editor d'ajout dans la scène ouverte.

Règle importante : n'utilisez pas la commande qui reconstruit une scène. Toute modification doit être faite dans **Cours3_RobotCollecteur**.

Étape 1 Préparer la scène existante 5 minutes

- 1 Ouvrez le prototype avec Unity 6000.2.5f1 et attendez la fin de la compilation.
- 2 Ouvrez Assets/Scenes/Cours3_RobotCollecteur.unity.
- 3 Lancez la scène et vérifiez le déplacement, les trois batteries, les obstacles et la porte.
- 4 Créez un objet vide nommé Ennemis. Conservez les arrière-plans et la Tilemap. L'interface sera construite à l'étape 8.
- 5 Créez un commit Git avant de poursuivre.

Test de contrôle

Le jeu doit encore fonctionner exactement comme au début avant l'ajout des ennemis. Si une fonctionnalité du TP1 ou du TP2 est brisée, corrigez-la avant de continuer.

Étape 2 Créer les deux ennemis 15 minutes

- 1 Importez Enemy.png comme Sprite 2D avec 100 pixels par unité et un filtrage Point.

- 2 Créez Ennemi_1 et Ennemi_2 comme enfants de l'objet Ennemis.
- 3 Attribuez le sprite Enemy.png aux deux SpriteRenderer.
- 4 Utilisez une échelle voisine de 0,13. Une échelle de 1 rend le sprite beaucoup trop grand pour la scène.
- 5 Ajoutez un Rigidbody2D sans gravité, bloquez la rotation et ajoutez un CapsuleCollider2D configuré comme Trigger.
- 6 Pour chaque ennemi, créez deux objets vides PointA et PointB afin de délimiter son trajet.

Objet	Position de départ suggérée	Position de fin suggérée	Type
Ennemi_1	(-4,5 ; 1,2)	(-0,5 ; 1,2)	Patrouille
Ennemi_2	(1,5 ; -2,1)	(6 ; -2,1)	Sinusoïdal

Résultat attendu

Les deux ennemis sont visibles au-dessus du décor, restent dans l'aire de jeu et ne déplacent pas physiquement le robot lors d'un contact.

Étape 3 Programmer des déplacements différents 30 minutes

Créez EnnemiMobile.cs. Un seul composant doit permettre de choisir le comportement dans l'Inspector.

```
public enum TypeDeplacement
{
    Patrouille,
    Sinusoïdal
}

[SerializeField] private TypeDeplacement typeDeplacement;
```

Comportement Patrouille

- Choisir PointB comme première cible.
- Calculer la direction normalisée vers la cible.
- Appliquer la vitesse au Rigidbody2D.
- Changer de cible lorsque l'ennemi arrive à moins de 0,2 unité.

Comportement Sinusoïdal

Faites progresser l'ennemi entre PointA et PointB avec Mathf.PingPong. Ajoutez ensuite un décalage vertical calculé avec Mathf.Sin. Exposez dans l'Inspector la hauteur et la fréquence de la vague.

```
float allerRetour = Mathf.PingPong(progression, 1f);
Vector2 baseTrajet = Vector2.Lerp(pointA.position, pointB.position, allerRetour);
Vector2 cible = baseTrajet + Vector2.up * Mathf.Sin(progression * Mathf.PI * 2f) *
hauteurVague;
```

Test

Lancez le jeu pendant au moins 20 secondes. Ennemi_1 doit effectuer un aller-retour horizontal. Ennemi_2 doit suivre une trajectoire ondulée clairement différente.

Étape 4 Ajouter la détection et la poursuite 15 minutes

Ajoutez une référence vers le Transform du joueur, un rayon de détection et une vitesse de poursuite. Quand le robot est suffisamment proche, le déplacement normal est temporairement remplacé par une poursuite.

```
bool joueurDetecte = joueur != null &&  
Vector2.Distance(corps.position, joueur.position) <= rayonDetection;
```

- 1 Utilisez une vitesse de poursuite supérieure à la vitesse normale.
- 2 Retournez au comportement initial quand le robot quitte le rayon.
- 3 Retournez visuellement le sprite selon la direction horizontale.
- 4 Arrêtez les ennemis lorsque GestionJeu indique que la partie est terminée.

Diagnostic visuel

Ajoutez OnDrawGizmosSelected pour afficher le rayon de détection et la ligne entre PointA et PointB. Les Gizmos doivent être visibles uniquement lorsque l'ennemi est sélectionné.

Étape 5 Gérer le contact avec le robot 15 minutes

- 1 Dans OnTriggerStay2D, vérifiez que l'objet possède le tag Player.
- 2 Ajoutez un délai entre deux impacts pour empêcher la perte instantanée de toutes les vies.
- 3 Appelez GestionJeu.Instance.PerdreVie().
- 4 Remplacez le robot sur PointDepart et annulez sa vitesse.
- 5 Vérifiez que la défaite s'affiche lorsque les vies atteignent zéro.

Valeur suggérée : 1,2 seconde entre deux impacts. Le joueur doit avoir le temps de reprendre le contrôle après son retour au point de départ.

Étape 6 Ajouter l'effet d'attaque 20 minutes

Créez EffetAttaqueEnnemi.cs. Cet effet ne remplace pas la logique de jeu : il rend simplement l'impact compréhensible pour le joueur.

Réteur	Comportement attendu	Durée suggérée
Ennemi	Couleur orange puis blanche et variation rapide d'échelle	0,18 s
Robot	Clignotement rouge et léger agrandissement	0,45 s
Caméra	Petit décalage aléatoire avec atténuation	0,14 s
Audio	Son bref joué une seule fois par impact	Moins de 0,4 s

Le script SecousseCamera doit produire un décalage temporaire lu par CameraSuivi. Remettez le décalage à zéro à la fin pour éviter que la caméra dérive.

Test

- Un seul contact produit une seule perte de vie.
- L'effet reste visible sans masquer l'interface.
- La caméra retrouve exactement son suivi normal après la secousse.

- Les effets peuvent être rejoués après le délai de protection.

Étape 7 Ajouter les sons et compléter la boucle 20 minutes

Créez un objet AudioJeu contenant le composant AudioJeu et deux AudioSource. La première source joue les effets ponctuels. La seconde joue l'ambiance en boucle avec un volume plus faible.

- 1 Vérifiez que la caméra principale possède exactement un AudioListener.
- 2 Configurez la source des effets avec Play On Awake désactivé et Spatial Blend à 0.
- 3 Configurez la source d'ambiance avec Loop activé, Spatial Blend à 0 et un volume voisin de 0,16.
- 4 Lancez l'ambiance dans Start et jouez un son de lancement au démarrage de GestionJeu.
- 5 Reliez ensuite les sons aux événements du tableau suivant.

Événement	Son attendu	Déclencheur
Lancement	Courte montée électronique	Start de GestionJeu
Ambiance	Bourdonnement électronique discret en boucle	Start de AudioJeu
Batterie	Collecte courte	AjouterBatterie
Objectif atteint	Signal positif	Troisième batterie
Impact	Signal bref	PerdreVie
Victoire	Séquence positive	DéclencherVictoire
Défaite	Séquence descendante	DéclencherDéfaite ou temps écoulé

La minuterie doit appeler la défaite lorsque le temps atteint zéro. La porte reste cachée jusqu'à la collecte des trois batteries. Après la victoire ou la défaite, les commandes du joueur et les déplacements des ennemis doivent s'arrêter.

Si aucun son ne se fait entendre

- Vérifiez que Mute Audio n'est pas activé dans la fenêtre Game.
- Vérifiez la présence d'un AudioListener sur Main Camera.
- Vérifiez que AudioJeu est assigné dans le composant GestionJeu.
- Vérifiez que les volumes des deux AudioSource et le volume général de Unity sont supérieurs à zéro.
- Relancez la scène après avoir sauvegardé les modifications.

Étape 8 Construire l'interface du jeu 35 minutes

Créez ou complétez un Canvas nommé Interface. Utilisez le mode Screen Space Overlay et un Canvas Scaler réglé sur Scale With Screen Size avec une résolution de référence de 1920 par 1080.

Element UI	Contenu attendu	Ancrage conseillé
Logo	Titre ou logo Robot Collecteur	Coin supérieur gauche
Énergie	Icône batterie, texte Batteries : 0/3 et barre de progression	Haut au centre

Élément UI	Contenu attendu	Ancrage conseillé
Intégrité	Icône cœur et texte Vies : 3	Haut à droite
Minuterie	Icône chronomètre et valeur 02:00	Coin supérieur droit
Consigne	Commandes et objectif principal	Bas au centre
Victoire	Titre, message et bouton Recommencer	Panneau centré plein écran
Défaite	Titre, message et bouton Recommencer	Panneau centré plein écran
Introduction	Logo, messages de mission, fondu et commande Passer	Plein écran au-dessus du HUD

- 1 Utilisez TextMeshPro pour tous les textes dynamiques.
- 2 Créez une barre de progression avec une Image de type Filled et Fill Method Horizontal.
- 3 Reliez texteBatteries, texteVies, barreProgression, panneauVictoire et panneauDefaite dans GestionJeu.
- 4 Reliez le texte du chronomètre dans MinuterieJeu.
- 5 Branchez les deux boutons Recommencer sur GestionJeu.RecommencerPartie.
- 6 Désactivez les panneaux de victoire et de défaite au démarrage.
- 7 Testez l'interface en formats 16:9 et 4:3. Aucun élément ne doit quitter l'écran.
- 8 Ajoutez le composant IntroductionJeu et reliez son CanvasGroup et son texte de mission.

Introduction cinématique

- 1 Créez un panneau noir plein écran avec un CanvasGroup et placez-le au-dessus du HUD.
- 2 Affichez successivement le logo, Station orbitale 07, Énergie critique et la mission de collecte.
- 3 Utilisez Time.unscaledDeltaTime et WaitForSecondsRealtime pour que les fondus fonctionnent pendant la pause.
- 4 Pendant l'introduction, mettez Time.timeScale à 0. Rétablissez-le à 1 à la fin.
- 5 Permettez au joueur de passer l'introduction avec Espace ou Échap.
- 6 La durée totale suggérée est de cinq secondes. Le son de lancement doit accompagner l'apparition du logo.

Règles de lisibilité

- Les valeurs importantes doivent rester lisibles au-dessus du décor.
- Utilisez des panneaux semi-opaques derrière les textes si nécessaire.
- La minuterie doit devenir orange sous 30 secondes et rouge sous 10 secondes.
- Les écrans de fin doivent empêcher le joueur de continuer à se déplacer.

Étape 9 Effectuer les tests finaux 25 minutes

Test	Action	Résultat attendu
TIntro	Lancer la scène sans appuyer sur une touche.	Logo, messages et fondus sont joués avant le début du jeu.
T0	Relancer puis appuyer sur Espace.	L'introduction est passée, le jeu reprend et l'ambiance est audible.
T1	Observer les ennemis sans approcher.	Les déplacements sont différents et restent dans leurs trajets.
T2	Entrer dans le rayon de chaque ennemi.	L'ennemi abandonne temporairement son trajet et poursuit le robot.
T3	Toucher un ennemi.	Effet, son d'impact, perte d'une vie et retour au départ.
T4	Rester près d'un ennemi.	Le délai empêche plusieurs pertes immédiates.
T5	Collecter les trois batteries.	Chaque collecte produit un son; la porte apparaît avec le signal d'objectif.
T5b	Observer le HUD pendant la collecte.	Le compteur et la barre de progression sont actualisés.
T5c	Changer le format de la fenêtre Game.	Le HUD reste correctement ancré et lisible.
T6	Atteindre la porte.	L'écran et le son de victoire se déclenchent.
T7	Perdre toutes les vies ou attendre 0:00.	L'écran et le son de défaite se déclenchent.
T8	Cliquer sur Recommencer.	La scène repart et l'ambiance est relancée.

Remise

- Lien vers le dépôt GitHub et accès accordé à l'enseignant comme collaborateur.
- Scène Cours3_RobotCollecteur mise à jour, sans nouvelle scène de remplacement.
- Scripts C# organisés dans les dossiers Scripts/Interactions, Scripts/Jeu et Scripts/Camera.
- Canvas complet avec introduction, HUD, minuterie, panneaux de fin et boutons fonctionnels.
- Historique Git contenant au moins les commits fonctionnels demandés.
- Courte capture vidéo ou démonstration en classe montrant les deux déplacements, la poursuite, l'impact et la fin de partie.

Critères d'évaluation

Critère	Points
Conservation et organisation de la scène existante	5
Deux déplacements distincts et configurables	20
Détection, poursuite et retour au comportement initial	10
Contact, vies, retour au départ et délai de protection	10
Effet d'attaque, secousse et audio	15
Introduction, HUD, minuterie, progression et écrans de fin	20
Boucle victoire, défaite, minuterie et redémarrage	10
Qualité du code, tests et commits Git	10
Total	100

Questions de compréhension

- 1 Pourquoi utilise-t-on un enum pour choisir le type de déplacement plutôt que deux scripts presque identiques?
- 2 Pourquoi faut-il ajouter un délai entre deux impacts dans OnTriggerStay2D?
- 3 Expliquez comment GestionJeu, EnnemiMobile et EffetAttaqueEnnemi collaborent lors d'un contact.

Défi facultatif

Ajoutez un troisième comportement au même enum, par exemple une orbite autour d'un point ou une patrouille entre trois positions. Le comportement doit rester configurable dans l'Inspector et ne pas modifier les deux ennemis obligatoires.